Download stb_image.h & stb_image.h from gcu learnand copy them into your project directory (same folder as all your other classes). Stb is used to load images for textures.

Right click on your "header files" folder in solution explorer and select add exiting file, then add stb_image.h. Do the same with the source files folder and add stb_image.h.

Compile the program, if you get a warning do the following:

Select your project and click "Properties" in the context menu.

In the dialog, chose Configuration Properties -> C/C++ -> Preprocessor

In the field PreprocessorDefinitions add ;_CRT_SECURE_NO_WARNINGS to turn those warnings off.

Create new class called Texture (right click the project, select add then class (Visual c++ class)). The texture class is going to be used to load a texture from a file and bind it. The header file is as follows

```
#pragma once
#include <string>
#include <GL\glew.h>

class Texture
{
public:
	Texture(const std::string& fileName);

	void Bind(unsigned int unit); // bind upto 32 textures

	~Texture();

protected:
private:

	GLuint textureHandler;
};
```

TODO:

Texture class:

Set up includes:

```
#include "Texture.h"
#include "stb_image.h"
#include <cassert>
#include <iostream>
```

Inside the constructor:

- create three integers to hold the width, height, and number of components of the image
- load the image from file using stb_load() and pass the addresses of the above integers to the program can write the data. The data is stored in "imageData"
    - `unsigned char* imageData = stbi_load((fileName).c_str(), addressofwidth, addressofheight, addressofcomponents, 4) //4 is the required components (no important for us)`
- check the image data loaded
    - ```
      if (imageData == NULL)
      {
          std::cerr << "texture load failed" << fileName << std::endl;
      }
      ```
- generate the buffer to store the texture in opengl
    - `glGenTextures(1, &textureHandler); // number of and address of textures`
- bind the texture
    - `glBindTexture(GL_TEXTURE_2D, textureHandler); //bind texture - define type & specify texture we are working with`
- Set the parameters to control texture wrapping and linear filtering
    - `glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT); // wrap texture outside width`
    - `glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT); // wrap texture outside height`
    - `glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR); // linear filtering for minification (texture is smaller than area)`
    - `glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR); // linear filtering for magnification (texture is larger)`
- send texture to the GPU
    - `glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA, width, height, 0, GL_RGBA, GL_UNSIGNED_BYTE, imageData); //Target, Mipmapping Level, Pixel Format, Width, Height, Border Size, Input Format, Data Type of Texture, Our Images Data`
- delete the data from CPU
    - `stbi_image_free(imageData);`

Inside the destructor:

- delete our textures
    - `glDeleteTextures(1, &textureHandler); // number of and address of textures`

In the bind method:

- check we are working with one of the 32 textures
    - `assert(unit >= 0 && unit <= 31); /// check we are working with one of the 32 textures`
- set active texture and bind it
    - `glActiveTexture(GL_TEXTURE0 + unit); //set acitve texture unit`
    - `glBindTexture(GL_TEXTURE_2D, textureHandler); //type of and texture to bind to unit`

Open up mesh.h and update as follows:

- Create a variable to store the position on the texture that we are mapping to the vertex
      ```

- glm::vec2 texCoord;
- change the constructor to take a vec2 in as an parameter and set texCoord to = the argument
- update our enum as follows:
  - ```
    enum
    {

        POSITION_VERTEXBUFFER,
        TEXCOORD_VB,
        NUM_BUFFERS

    };
    ```

We have now changed the vertex in a manner that our positions are no longer sequential. previously were only working with positions which were stored in out vec3 pos.

```
glGenBuffers(NUM_BUFFERS, vertexArrayBuffers);
glBindBuffer(GL_ARRAY_BUFFER, vertexArrayBuffers[POSITION_VERTEXBUFFER]);
glBufferData(GL_ARRAY_BUFFER, numVertices * sizeof(vertices[0]), vertices, GL_STATIC_DRAW);
```

This meant when were moving sequentially through our array of vertices we were ONLY working with pos (one buffer). However now we have added texCoord (a second buffer). So when we move sequentially through our array of vertices we will be moving like so (two buffers)...

pos
texCoord
pos
texCoord
pos
texCoord

We therefore have to update the mesh class to date this into consideration.

Open up mesh.cpp and update as follows:

- create two lists of vectors to hold our buffer data
  - std::vector<glm::vec3> positions; //holds the position data
  - std::vector<glm::vec2> textCoords; //holds the texture coordinate data
- reserve space for our lists
  - positions.reserve(numVertices); // reserve the all the space needed to hold our data
  - textCoords.reserve(numVertices);
- Create a for loop that runs to number of vertices and stores the data in our lists, use:
  - positions.push_back(vertices[i].pos); //store our array of vertex positon into a list vec3 positions
  - textCoords.push_back(vertices[i].texCoord);
```

- setup our two buffers as follows, this is the same as the first time you set them up only this time we have two:

```
glGenBuffers(NUM_BUFFERS, vertexArrayBuffers); //generate our buffers based of our array of data/buffers - GLuint vertexArrayBuffers[NUM_BUFFERS];
glBindBuffer(GL_ARRAY_BUFFER, vertexArrayBuffers[POSITION_VERTEXBUFFER]); //tell opengl what type of data the buffer is (GL_ARRAY_BUFFER), and pass the data
glBufferData(GL_ARRAY_BUFFER, numVertices * sizeof(positions[0]), &positions[0], GL_STATIC_DRAW); //move the data to the GPU - type of data, size of data, starting address (pointer) of data, where do we store the data on the GPU

glEnableVertexAttribArray(0);
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 0, 0);

glBindBuffer(GL_ARRAY_BUFFER, vertexArrayBuffers[TEXCOORD_VB]); //tell opengl what type of data the buffer is (GL_ARRAY_BUFFER), and pass the data
glBufferData(GL_ARRAY_BUFFER, numVertices * sizeof(textCoords[0]), &textCoords[0], GL_STATIC_DRAW); //move the data to the GPU - type of data, size of data, starting address (pointer) of data, where do we store the data on the GPU

glEnableVertexAttribArray(1);
glVertexAttribPointer(1, 2, GL_FLOAT, GL_FALSE, 0, 0);

glBindVertexArray(0); // unbind our VAO
```

Now we need to update the vertices in the MainGame as they take two parameters and we are only passing in one:

In your draw game method change as follows:

```
Vertex vertices[] = {   Vertex(glm::vec3(-0.5, -0.5, 0), glm::vec2(0.0, 0.0)),
                        Vertex(glm::vec3(0, 0.5, 0), glm::vec2(0.5, 1.0)),
                        Vertex(glm::vec3(0.5, -0.5, 0), glm::vec2(1.0, 0.0)) };
```

Open your two shaders and change as follows**, copy and paste this code,** note the **position, texCoord,** and **transform** variable which we will be linking from in our shader class:

Vertex:

```
#version 120

attribute vec3 position;
attribute vec2 texCoord;

varying vec2 texCoord0;

uniform mat4 transform;
```

```
void main()
{
	gl_Position = transform * vec4(position, 1.0);
	texCoord0 = texCoord;
}
```

Fragment:

```
#version 120

varying vec2 texCoord0;

uniform sampler2D diffuse;

void main()
{
	gl_FragColor = texture2D(diffuse, texCoord0);
}
```

Add a new header file and call it Transform. This is a struct that hold simple math which we discussed in the last lecture, again you **can copy and paste this code**:

```
#pragma once

#include <glm/glm.hpp>
#include <glm/gtx/transform.hpp>
#include "camera.h"

struct Transform
{
public:
	Transform(const glm::vec3& pos = glm::vec3(), const glm::vec3& rot = glm::vec3(), const glm::vec3& scale = glm::vec3(1.0f, 1.0f, 1.0f))
	{
		this->pos = pos;
		this->rot = rot;
		this->scale = scale;
	}

	inline glm::mat4 GetModel() const //runs as compile time
	{
		glm::mat4 posMat = glm::translate(pos);
		glm::mat4 scaleMat = glm::scale(scale);
		glm::mat4 rotX = glm::rotate(rot.x, glm::vec3(1.0, 0.0, 0.0));
		glm::mat4 rotY = glm::rotate(rot.y, glm::vec3(0.0, 1.0, 0.0));
		glm::mat4 rotZ = glm::rotate(rot.z, glm::vec3(0.0, 0.0, 1.0));
		glm::mat4 rotMat = rotX * rotY * rotZ;

		return posMat * rotMat * scaleMat;
	}
```

```
	inline glm::vec3* GetPos() { return &pos; } //getters
	inline glm::vec3* GetRot() { return &rot; }
	inline glm::vec3* GetScale() { return &scale; }

	inline void SetPos(glm::vec3& pos) { this->pos = pos; } // setters
	inline void SetRot(glm::vec3& rot) { this->rot = rot; }
	inline void SetScale(glm::vec3& scale) { this->scale = scale; }
protected:
private:
	glm::vec3 pos;
	glm::vec3 rot;
	glm::vec3 scale;
};
```

We are going to be running this maths on the GPU for speed and efficiency, in order to run code on the GPU we need to access our shader, i.e. we are going to need to update our shader class.

open the shader.h and copy and paste this code, we are simply adding an enumeration and a GLuint uniforms, the enumeration hold the details of our uniforms which are shader variables (don't worry about these for now we are going to covert them in today's lecture):

```
#pragma once
#include <string>
#include <GL\glew.h>
#include "transform.h"

class Shader
{
public:
	Shader(const std::string& filename);

	void Bind(); //Set gpu to use our shaders
	void Update(const Transform& transform);

	std::string Shader::LoadShader(const std::string& fileName);
	void Shader::CheckShaderError(GLuint shader, GLuint flag, bool isProgram, const
std::string& errorMessage);
	GLuint Shader::CreateShader(const std::string& text, unsigned int type);

  ~Shader();


protected:
private:
	static const unsigned int NUM_SHADERS = 2; // number of shaders

	enum
	{
		TRANSFORM_U,

		NUM_UNIFORMS
```

```
	};

	GLuint program; // Track the shader program
	GLuint shaders[NUM_SHADERS]; //array of shaders
	GLuint uniforms[NUM_UNIFORMS]; //no of uniform variables
};
```

Now open the shader.cpp:

TODO

- associate our new texCoord attribute with our shader (after our position att)
    - glBindAttribLocation(program, 1, "texCoord");
- setup the uniform with the shader program
    - uniforms[TRANSFORM_U] = glGetUniformLocation(program, "transform");

Create and update method to update the transforms:

```
void Shader::Update(const Transform& transform)
{
	glm::mat4 model = transform.GetModel();
	glUniformMatrix4fv(uniforms[TRANSFORM_U], 1, GLU_FALSE, &model[0][0]);
}
```

Now we can skin, draw and move our triangle update the MainGame drawGame function as follow:

```
void MainGame::drawGame()
{
	_gameDisplay.clearDisplay();

	Vertex vertices[] = {Vertex(glm::vec3(-0.5, -0.5, 0), glm::vec2(0.0, 0.0)),
				Vertex(glm::vec3(0, 0.5, 0), glm::vec2(0.5, 1.0)),
				Vertex(glm::vec3(0.5, -0.5, 0), glm::vec2(1.0, 0.0)) };

	Mesh mesh(vertices, sizeof(vertices) / sizeof(vertices[0])); //size calcuated by number of
bytes of an array / no bytes of one element

	Shader shader(".\\res\\shader"); //new shader
	Texture texture("C:\\Users\\Admin\\Documents\\Visual Studio
2013\\Projects\\Lab3\\res\\bricks.jpg"); //load texture
	Transform transform;
	transform.SetPos(glm::vec3(sinf(counter), 0.0, 0.0));
	transform.SetRot(glm::vec3(0.0, 0.0, counter * 5));
	transform.SetScale(glm::vec3(sinf(counter), sinf(counter), sinf(counter)));

	shader.Bind();
	shader.Update(transform);
	texture.Bind(0);
	mesh.Draw();

	counter = counter + 0.01f;
```

```
        glEnableClientState(GL_COLOR_ARRAY);
        glEnd();

        _gameDisplay.swapBuffer();
}
```

TODO

add another triangle, skin it with the water.jpg in your res folder and move it in the opposite direction from your bricks one.